CSL7530: Advanced Machine Learning
<u>Assignment:2</u>

Group members:
1. Krishna Balaji Patil (B22CS078)
2. Harshit Goyal (B22CS024)
3. Saumitra Agrawal (B22AI054)

Code can be found at: https://github.com/kp-19/CSL7530_AML/tree/main/Assignment2

Code files details:
GPRegression.py : Implementation of algorithm 2.1 from scratch
GPClassification.py : Implementation of algorithm 3.1 and 3.2 from scratch
california_housing_regression.py : GP applied to california housing dataset.
breast_cancer_classification.py : GP classifier applied to breast cancer dataset.
toy_dataset.py : 2D toy dataset for testing GP regression.

Part 1- Gaussian Process Regression

1. Introduction:

Gaussian Processes (GPs) is a non-parametric Bayesian approach for regression. Instead of learning explicit model parameters like linear regression or neural networks, a GP defines a distribution over functions. Predictions are made by conditioning this distribution on observed training data.

A Gaussian Process is fully defined by:

- a **mean function** m(x)
- a **covariance function (kernel)** k(x,x′)

In this assignment, **Algorithm 2.1 from** Gaussian Processes for Machine Learning by Carl Edward Rasmussen and Christopher K. I. Williams was implemented to perform Gaussian Process Regression.

The goal is to compute the **predictive mean and predictive variance** for unseen inputs.

2. Kernel function:

The implementation is using the RBF kernel (Radial Basis Function kernel) which is also known as the Gaussian kernel.

$$k(x,y) = \exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right)$$

(Here, l is length scale, σ_f^2 is signal variance, and $||x_i-x_j||^2$ is squared euclidean distance)

The RBF kernel assumes that **points closer in feature space produce more similar outputs**. It produces smooth functions and is widely used in Gaussian Process models.

3. Implementation details:

Algorithm 2.1 was implemented from scratch as the function ***GPRegression*** with the help of Python's numpy library in the file GPRegression.py. The RBF kernel was implemented with the following hyper-parameters:

- Scale-length = 1.0
- Variance (sigma) = 1.0

The function GPRegression was implemented with default noise variance of 0.1. It returns the predictive mean (f_star), predictive variance (cov) and the log likelihood for the given test point.

4. Toy Dataset Experiment:

A synthetic dataset was generated in a **two dimensional input space**.

Training samples were created by sampling random points in the range:

$$x1,x2 \in [-3,3]$$

The true function used was:

$$f(x)=\sin(x_1^2+x_2^2)$$

Gaussian noise with standard deviation 0.1 was added to simulate measurement noise.

4.1 Prediction for a Test Input:

For a test point:

$x*=[0.5,-1.0]$

The model produced:

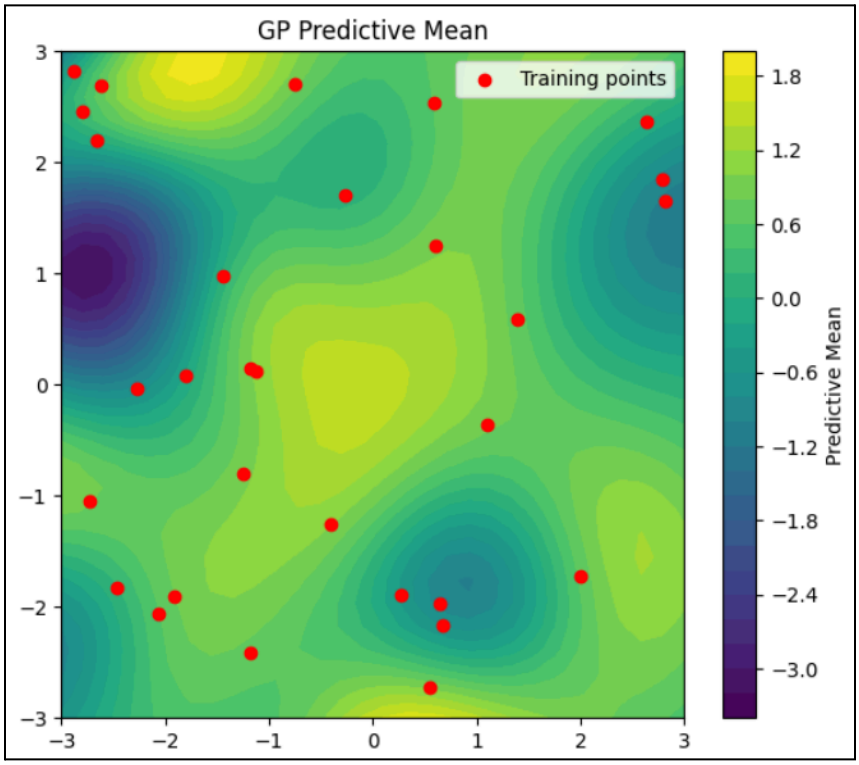
Predictive Mean: 0.3359

Predictive Variance: 0.1129

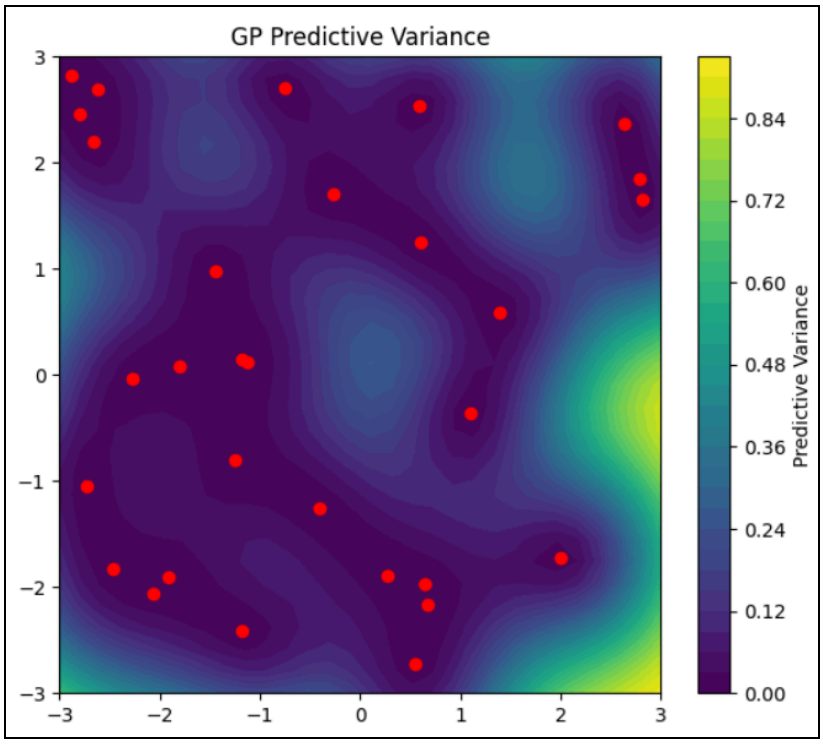
The predictive mean represents the expected value of the function at the test point, while the variance reflects the uncertainty of the prediction.

4.2 Visualization:

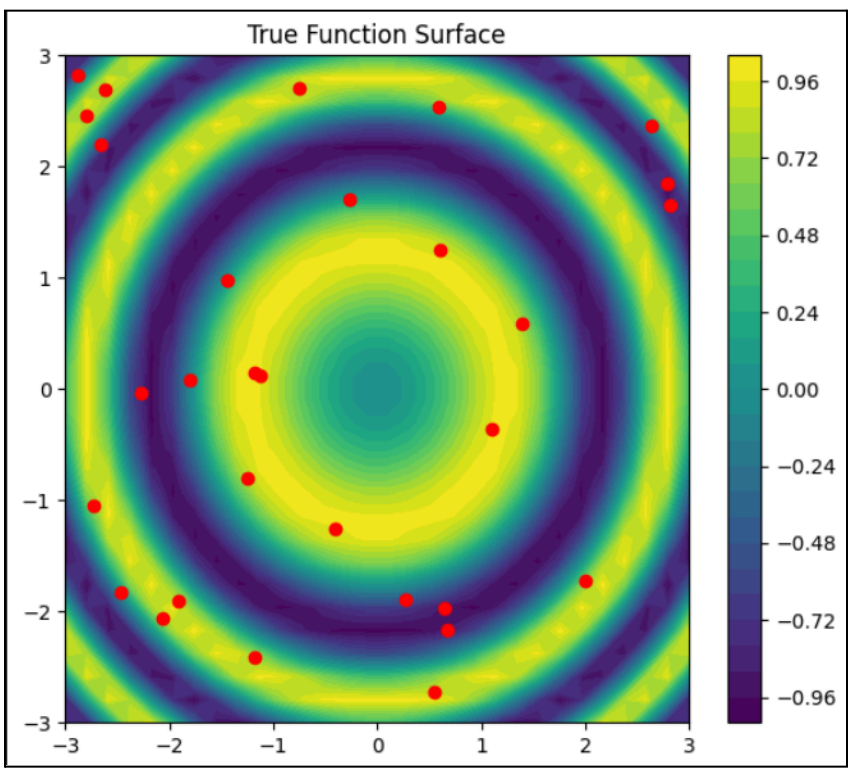
1. GP Predictive mean: The following plot shows the function learnt by the Gaussian Process. This is the Predictive Mean surface and captures the general structure of the true function.



2. GP predictive variance: The following plot shows the GP predictive variance and captures the uncertainty across the input space. Key observations:
   - Variance is **low near training points**
   - Variance increases **far away from observed data**



3. True function surface: The true function surface is plotted to compare with the GP prediction and it can be seen that the GP approximation closely matches the true function in regions where training data is available.



5. Regression on California Housing Dataset:

To evaluate the GP regression model on a real dataset, the California Housing dataset from Scikit-learn was used.

5.1 Dataset characteristics:

- Number of samples: 20640
- Number of features: 8

The dataset predicts median house value using demographic and geographical features.

5.2 Preprocessing:

- The following preprocessing steps were performed:
- Feature scaling using StandardScaler
- Train–test split with 80% training data and 20% testing data

Due to the cubic complexity of Gaussian Processes the training data was limited to 1000 samples and test data to 200 samples.

5.3 Regression results:

Mean squared error	0.5247
Root Mean Squared error	0.7243
R^2 score	0.5708

Interpretation:

- RMSE indicates the average prediction error.
- The R^2 score of 0.57 indicates the model explains a moderate portion of the variance in house prices.

Predicted mean for the first 10 test points:

[[0.37747157], [0.62194701], [3.04789013], [2.60960315], [2.93969456], [1.67304143], [2.47251672], [1.71712589], [2.41483058], [5.1002827]]

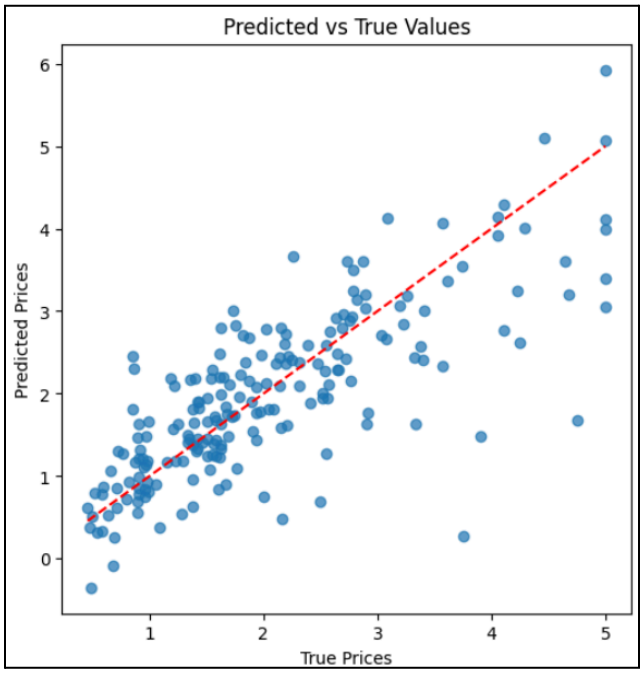
Diagonal values of variance matrix:

[0.03764168, 0.02549267, 0.02080728, 0.00667305, 0.01246195, 0.03171455,

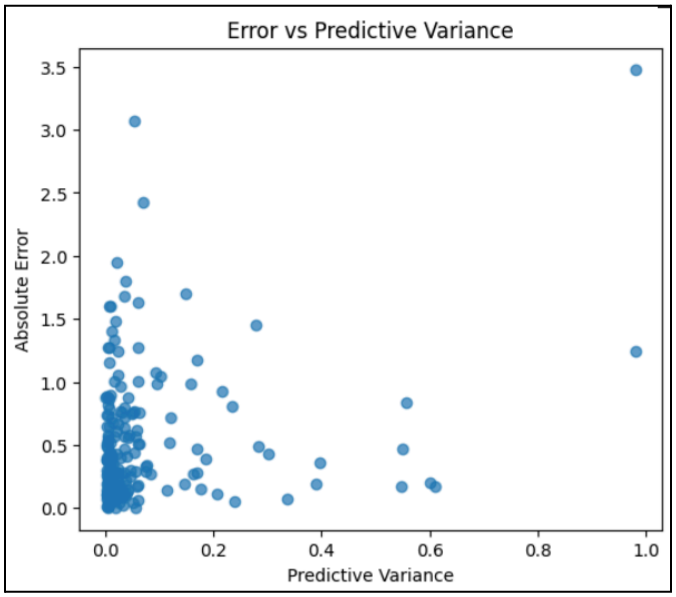
0.00209633, 0.00150768, 0.09629011, 0.03524161, …….]

5.4 Visualization:

1. **Predicted vs True Prices**: This scatter plot compares predicted house prices with actual prices.



2. **Predictive Variance vs Absolute Error**: This plot compares the model's predicted uncertainty with the actual prediction error.



Part 2- Gaussian Process Classification

1. Introduction:

Gaussian Processes (GPs) provide a probabilistic approach to machine learning where predictions are expressed as distributions rather than single deterministic values. In classification tasks, Gaussian Process Classification (GPC) models a latent function f(x) using a Gaussian process prior and transforms it into class probabilities using a likelihood function such as the logistic or probit function.

Unlike standard classifiers that directly predict labels, GPC provides predictive uncertainty, which is useful for understanding how confident the model is about its predictions.

However, because the likelihood function for classification is non-Gaussian, the posterior distribution cannot be computed analytically. Therefore, an approximation method is required. In this work, the Laplace approximation is used, which approximates the posterior distribution around its mode.

The implementation follows:

- **Algorithm 3.1:** Newton method to find the posterior mode f_hat

input: K (covariance matrix), $\mathbf{y}$ (± 1 targets), $p(\mathbf{y}|\mathbf{f})$ (likelihood function)
2: $\mathbf{f} := \mathbf{0}$ — initialization
repeat — Newton iteration
4: $W := -\nabla\nabla \log p(\mathbf{y}|\mathbf{f})$ — eval. W e.g. using eq. (3.15) or (3.16)
$L := \text{cholesky}(I + W^{\frac{1}{2}} K W^{\frac{1}{2}})$ — $B = I + W^{\frac{1}{2}} K W^{\frac{1}{2}}$
6: $\mathbf{b} := W\mathbf{f} + \nabla \log p(\mathbf{y}|\mathbf{f})$
$\mathbf{a} := \mathbf{b} - W^{\frac{1}{2}} L^\top \backslash (L \backslash (W^{\frac{1}{2}} K \mathbf{b}))$ } eq. (3.18) using eq. (3.27)
8: $\mathbf{f} := K\mathbf{a}$
until convergence — objective: $-\frac{1}{2}\mathbf{a}^\top \mathbf{f} + \log p(\mathbf{y}|\mathbf{f})$
10: $\log q(\mathbf{y}|X, \theta) := -\frac{1}{2}\mathbf{a}^\top \mathbf{f} + \log p(\mathbf{y}|\mathbf{f}) - \sum_i \log L_{ii}$ — eq. (3.32)
return: $\hat{\mathbf{f}} := \mathbf{f}$ (post. mode), $\log q(\mathbf{y}|X, \theta)$ (approx. log marg. likelihood)

Algorithm 3.1: Mode-finding for binary Laplace GPC. Commonly used convergence criteria depend on the difference in successive values of the objective function $\Psi(\mathbf{f})$ from eq. (3.12), the magnitude of the gradient vector $\nabla\Psi(\mathbf{f})$ from eq. (3.13) and/or the magnitude of the difference in successive values of $\mathbf{f}$. In a practical implementation one needs to secure against divergence by checking that each iteration leads to an increase in the objective (and trying a smaller step size if not). The computational complexity is dominated by the Cholesky decomposition in line 5 which takes $n^3/6$ operations (times the number of Newton iterations), all other operations are at most quadratic in n.

- **Algorithm 3.2:** Prediction of class probabilities for new test inputs

The model is implemented from scratch and evaluated on the **Breast Cancer dataset** for binary classification.

input: $\hat{\mathbf{f}}$ (mode), X (inputs), $\mathbf{y}$ (± 1 targets), k (covariance function), $p(\mathbf{y}|\mathbf{f})$ (likelihood function), $\mathbf{x}_*$ test input

2: $W := -\nabla\nabla \log p(\mathbf{y}|\hat{\mathbf{f}})$

$L := \text{cholesky}(I + W^{\frac{1}{2}} K W^{\frac{1}{2}})$ $\qquad B = I + W^{\frac{1}{2}} K W^{\frac{1}{2}}$

4: $\bar{f}_* := \mathbf{k}(\mathbf{x}_*)^\top \nabla \log p(\mathbf{y}|\hat{\mathbf{f}})$ $\qquad$ eq. (3.21)

$\mathbf{v} := L \backslash \left(W^{\frac{1}{2}} \mathbf{k}(\mathbf{x}_*)\right)$

6: $\mathbb{V}[f_*] := k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{v}^\top \mathbf{v}$ $\qquad \Big\}$ eq. (3.24) using eq. (3.29)

$\bar{\pi}_* := \int \sigma(z) \mathcal{N}(z|\bar{f}_*, \mathbb{V}[f_*]) dz$ $\qquad$ eq. (3.25)

8: **return**: $\bar{\pi}_*$ (predictive class probability (for class 1))

Algorithm 3.2: Predictions for binary Laplace GPC. The posterior mode $\hat{\mathbf{f}}$ (which can be computed using Algorithm 3.1) is input. For multiple test inputs lines $4-7$ are applied to each test input. Computational complexity is $n^3/6$ operations once (line 3) plus n^2 operations per test case (line 5). The one-dimensional integral in line 7 can be done analytically for cumulative Gaussian likelihood, otherwise it is computed using an approximation or numerical quadrature.

2. Kernel function:

Similar to the regression task, for classification as well, we have used the Radial Basis Function (RBF) kernel.

The RBF kernel is widely used because it assumes smooth functions and can model complex nonlinear relationships between inputs and outputs.

3. Implementation steps:

All implementation was done with help of python's numpy library for mathematical and linear algebra operations. (GPClassification.py)

The kernel function was implemented with the hyper-parameters:

- Scale length = 1.0
- Variance (sigma) = 1.0

Algorithm 3.1 was implemented for calculating the posterior mode f_hat using the Newton's update as the function: ***compute_fhat*** . Tolerance value for terminating the loop was taken as 10^{-6} and max iterations = 50. The function returns posterior mode (f_hat), intermediate matrix a, log-likelihood and the marginal log-likelihood.

Algorithm 3.2 was implemented as the function ***GPClassification.*** It returns classification probabilities pi_star, predictive mean f_bar and predictive variance var.

4. Classification on Breast Cancer Dataset:

The model was evaluated on the Breast Cancer Wisconsin dataset, which is a binary classification dataset commonly used in machine learning experiments.

Dataset Details

- Total samples: 569
- Features: 30
- Classes:
    - Malignant
    - Benign

The dataset was split into 80% training and 20% test data.

Before training, the features were standardized using StandardScaler to ensure that all features have zero mean and unit variance.

Model Results

Initial values of f_hat : [0.40105814, -0.40106359, 0.40210262, 0.50115392, 0.40331536, -0.41471446, 0.40105853, 1.66250751, 1.76169382, -0.62010626, 0.73118727, -0.4890193, ……]

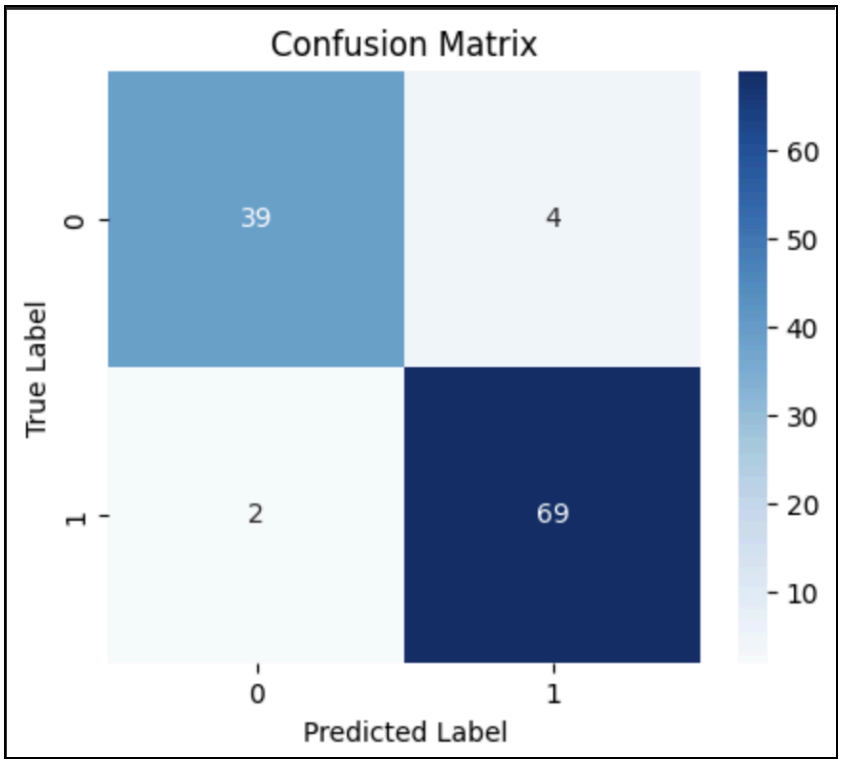
The model achieved the following performance:

Accuracy = 0.947

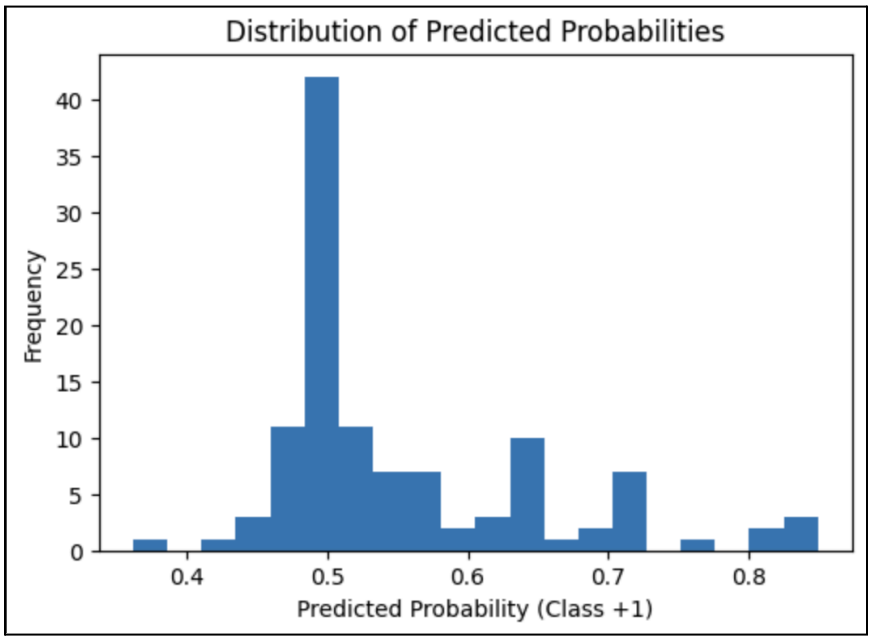
Classification Report

Class	Precision	Recall	F1-score
-1	0.95	0.91	0.93
1	0.95	0.97	0.96

The confusion matrix shows that the model correctly classified most samples, with only a few misclassifications.



Uncertainty in Predictions



One of the advantages of Gaussian Process Classification is that it provides **uncertainty estimates** through the predictive variance.

The histogram of predicted probabilities shows that:

- Many predictions cluster near **0.5**
- Some predictions are closer to **0.7–0.85**

Predictions close to **0.5** indicate **high uncertainty**, meaning the model is less confident about those points. These typically occur when:

- A data point lies near the **decision boundary**
- The test input is **far from training samples**
- The model cannot clearly separate the classes.

Predictions closer to **0 or 1** indicate **high confidence** in the classification.This probabilistic interpretation is a key advantage of Gaussian Processes compared to deterministic classifiers.